



UNIVERSITÀ
DI TORINO

A DENSITY BASED ALGORITHM FOR DISCOVERING CLUSTERS

Alessandro Zappatore

UNIVERSITÀ DEGLI STUDI DI TORINO

10/07/2026

DBSCAN: Un Algoritmo di Clustering Basato sulla Densità per Database Spaziali

- Le applicazioni moderne raccolgono enormi quantità di dati spaziali, rendendo cruciale la scoperta automatica della conoscenza.
- Il focus del paper è la **class identification**, ovvero il raggruppamento di oggetti di un database in sottoclassi significative.
- Viene presentato l'algoritmo **DBSCAN**, progettato per scoprire cluster di forma arbitraria e gestire il rumore.

Le Sfide del Clustering Spaziale

L'applicazione di algoritmi di clustering a grandi database spaziali richiede il soddisfacimento di tre requisiti fondamentali:

- **Minima conoscenza del dominio:** Spesso non si conoscono in anticipo i parametri appropriati del dominio di riferimento per settare l'algoritmo.
- **Forme arbitrarie:** I cluster in ambito spaziale possono avere forme di ogni tipo (sferiche, allungate, lineari, ecc.).
- **Ottima efficienza:** L'algoritmo deve funzionare su database di grandi dimensioni (molto più di qualche migliaio di oggetti).

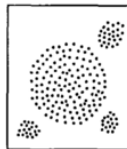
Analisi degli Algoritmi di Clustering Tradizionali

Gli algoritmi esistenti non soddisfano contemporaneamente i tre requisiti fondamentali.

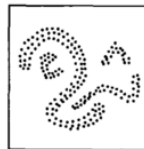
- **Algoritmi Partizionali** (es. **CLARANS**): Richiedono il numero di cluster k come input (conoscenza non sempre disponibile), tendono a trovare solo cluster convessi e hanno tempi di esecuzione proibitivi su larghi database.
- **Algoritmi Gerarchici** (es. **Ejcluster**): Non richiedono l'inserimento di k o conoscenza del dominio e trovano forme non convesse, ma richiedono la definizione di una condizione di terminazione e soffrono di un costo computazionale pari a $O(n^2)$, inaccettabile per grandi dataset.

L'Intuizione alla Base di DBSCAN

- DBSCAN riconosce i cluster basandosi sulla **densità tipica** dei punti.
- All'interno di ogni cluster, la densità dei punti è considerevolmente più alta rispetto all'esterno.
- Le aree di "rumore" (noise) sono caratterizzate da una densità di punti inferiore rispetto alla densità presente in qualsiasi cluster.



database 1

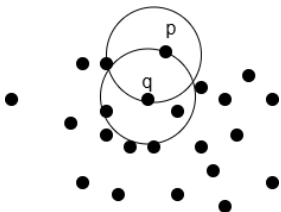


database 2



database 3

Definire la Densità Matematicamente



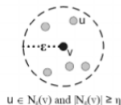
L'idea chiave è che per ogni punto in un cluster, un **intorno (neighborhood)** di un dato raggio deve contenere un numero minimo di punti (soglia di densità).

- **Eps-neighborhood** ($N_{Eps}(p)$): L'insieme dei punti che si trovano a una distanza minore o uguale a un raggio Eps dal punto p .
- **Core Point**: Un punto q il cui Eps-neighborhood contiene almeno un numero minimo di punti, definito dal parametro $MinPts$.

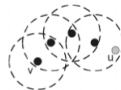
Le Relazioni tra Punti: Reachability e Connectivity

- **Directly density-reachable:** Un punto p è direttamente raggiungibile da un *core point* q se p è all'interno dell'Eps-neighborhood di q . È asimmetrico se p è un punto di bordo (border point).
- **Density-reachable:** Un punto p è raggiungibile in densità da q se esiste una "catena" di core points che li collega. Anche questa relazione è transitiva ma non simmetrica.
- **Density-connected:** Per collegare tra loro due border point dello stesso cluster, viene introdotta questa relazione simmetrica. Due punti p e q sono density-connected se sono entrambi density-reachable da un terzo punto o .

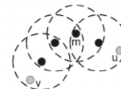
u is
directly density
reachable
from v



u is
density
reachable
from v



u is
density
connected
from v



Cos'è un Cluster per DBSCAN?

- Un **Cluster** è un sottoinsieme non vuoto di punti del database che soddisfa due condizioni fondamentali:
 - ① **Massimalità**: Se un punto p appartiene al cluster e q è *density-reachable* da p , allora anche q fa parte del cluster.
 - ② **Connettività**: Presi due punti qualsiasi p e q all'interno del cluster, essi devono essere *density-connected*.
- **Rumore (Noise)**: L'insieme di tutti i punti nel database che non appartengono a nessun cluster definito.
- Ogni punto all'interno del cluster è raggiungibile da qualsiasi *core point* del cluster.

Validazione dell'Algoritmo: Lemma 1 e Lemma 2

Questi due lemmi dimostrano la correttezza dell'approccio di DBSCAN nel trovare i cluster.

- **L'intuizione:** È possibile scoprire un intero cluster in due passi: scegliere un *core point* arbitrario come "seme" e recuperare tutti i punti *density-reachable* da esso.
- **Lemma 1:** Se p è un *core point* (cioè $|N_{Eps}(p)| \geq MinPts$), allora l'insieme O di tutti i punti raggiungibili in densità da p forma un **cluster** valido.
- **Lemma 2:** Dato un cluster C e un suo qualsiasi *core point* p , C è **esattamente uguale** all'insieme dei punti raggiungibili in densità da p .
- **Conseguenza:** Un cluster è determinato in modo univoco da *uno qualsiasi* dei suoi *core points*. Di conseguenza, l'ordine con cui l'algoritmo visita i punti non altera la composizione finale del cluster.

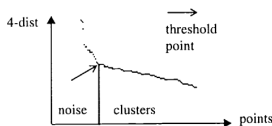
Come Funziona l'Algoritmo DBSCAN

- L'algoritmo inizia selezionando un punto arbitrario p nel database e recupera tutti i punti a esso density-reachable utilizzando i parametri globali Eps e $MinPts$.
- Se p è un *core point*, l'algoritmo forma un cluster.
- Se p è un *border point*, non ci sono punti raggiungibili e DBSCAN passa semplicemente al punto successivo.
- L'algoritmo utilizza la funzione `ExpandCluster` per espandere il cluster partendo da un punto ("seed"), aggiungendo progressivamente tutti i punti raggiungibili.

```
ExpandCluster(SetOfPoints, Point, ClId, Eps, MinPts)
: Boolean;
seeds:=SetOfPoints.regionQuery(Point,Eps);
IF seeds.size<MinPts THEN // no core point
    SetOfPoint.changeClId(Point,NOISE);
    RETURN False;
ELSE // all points in seeds are density-reachable
    SetOfPoints.changeClIds(seeds,ClId);
    seeds.delete(Point);
    WHILE seeds <> Empty DO
        currentP := seeds.first();
        result := SetOfPoints.regionQuery(currentP, Eps);
        IF result.size >= MinPts THEN
            FOR i FROM 1 TO result.size DO
                resultP := result.get(i);
                IF resultP.ClId IN {UNCLASSIFIED, NOISE} THEN
                    IF resultP.ClId = UNCLASSIFIED THEN
                        seeds.append(resultP);
                    END IF;
                    SetOfPoints.changeClId(resultP,ClId);
                END IF;
            END FOR;
        END IF;
        seeds.delete(currentP);
    END WHILE;
    RETURN True;
END IF
END; // ExpandCluster
```

- Il recupero dei punti nell'Eps.Neighborhood è gestito tramite region queries.
- Per garantire un'elaborazione rapida nei database spaziali, le region queries sono supportate da metodi di accesso spaziale come gli R*-trees.
- Grazie all'altezza limitata dell'R*-tree ($O(\log n)$), il tempo di esecuzione medio di una singola query è $O(\log n)$.
- La **complessità temporale complessiva** media di DBSCAN su un database di n punti è quindi $O(n \log n)$.

Un'Euristica per Scegliere Eps e MinPts

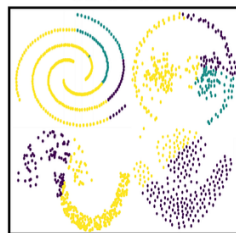


- É difficile conoscere in anticipo *Eps* e *MinPts* ottimali per il cluster meno densi ("thinnest cluster").
- Si utilizza la funzione **k-dist**, che mappa ogni punto alla distanza dal suo k-esimo vicino più prossimo. Ordinando in modo decrescente questi valori si ottiene un sorted k-dist graph.
- Viene proposto un approccio interattivo: l'utente cerca graficamente la prima "valle" o punto di flesso nel grafico, definendo come punto di soglia (threshold point).
- Il valore 4-dist del punto di soglia diventa il parametro *Eps*, e *MinPts* viene fissato a 4 per tutti i database bidimensionali, eliminando un parametro per l'utente.

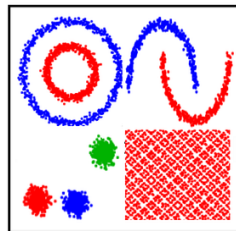
Accuratezza e Capacità di Trovare Forme Arbitrarie

- DBSCAN viene confrontato visivamente con l'algoritmo spaziale concorrente CLARANS su 3 database sintetici.
- DBSCAN rileva **tutti** i cluster corretti di forma arbitraria (convessa e non convessa) e individua senza problemi i punti di rumore.
- Al contrario, CLARANS divide in modo errato i cluster troppo grandi o troppo vicini ad altri, e non ha una vera nozione del rumore (assegna semplicemente i punti rumorosi al centroide più vicino).

CLARANS



DBSCAN



Test di Efficienza su SEQUOIA 2000

- Per testare l'efficienza su dati reali, gli autori hanno utilizzato il benchmark SEQUOIA 2000 contenente oltre 62000 punti geografici della California.
- Mentre il tempo di esecuzione di CLARANS cresce in modo quasi quadratico rispetto al numero di punti, quello di DBSCAN è solo leggermente superiore alla linearità.
- I risultati mostrano che DBSCAN è da 250 a 1900 **volte più veloce** di CLARANS.

Number of points	1252	2503	3910	5213	6256	7820	8937	10426	12512
DBSCAN	3.1	6.7	11.3	16.0	17.8	24.5	28.2	32.7	41.7
CLARANS	758	3026	6845	11745	18029	29826	39265	60540	80638